COFA/PERFORMANCE-TRACKING-TIMES · V2

Frontend Timing Setup

Our Laravel API records how long each request takes. The server can time its own work, but only your app knows when a request left and when its response arrived. This page shows the small addition that sends those two timestamps back, so the dashboard can show the full journey.



Calculating · your app Network out · app + server Delay · server Service · server Network return · app + server

Replace `https://api.example.com` with your API's address and `https://app.example.com` with your web frontend's address in every snippet below.

API owner On the Laravel API

Do this once, before the developers start. It lets browsers on the frontend's domain read the timing header.

1. In the API's `.env`, name the frontend's address. For several frontends, separate them with commas.

```
.env
PERFORMANCE_TRACKING_ALLOW_ORIGIN=https://app.example.com
```

2. Reload the configuration:

```
terminal
php artisan config:clear
```

3. The frontend must already be allowed in the API's CORS settings (`config/cors.php`, `allowed_origins`). If the app can call the API today, it is. Mobile apps need nothing here.

Web developer

In a web frontend

React, Vue, Angular, Svelte, Next.js, or plain JavaScript. One script tag, no npm package, no changes to your API calls.

The script never changes your requests or responses, and it sends its reports in the background with `navigator.sendBeacon`. Each report holds only a request id and timestamps.

1. Add this tag to the HTML template's `<head>`, **before** your app's own scripts. It wraps `fetch` and `XMLHttpRequest`, so axios, ky, jQuery and plain fetch are all covered.

   index.html

   ```
   <script src="https://api.example.com/performance-tracking/client.js"
           data-endpoint="https://api.example.com/performance-tracking/api/client"></script>
   ```

   With Next.js, add it in the root layout instead:

   app/layout.tsx

   ```
   import Script from 'next/script';

   // inside <html> … <body>
   <Script
     src="https://api.example.com/performance-tracking/client.js"
     data-endpoint="https://api.example.com/performance-tracking/api/client"
     strategy="beforeInteractive"
   />
   ```

2. If your site sends a Content-Security-Policy header, allow the API in `script-src` (to load the script). `connect-src` already allows it if the app calls the API.

3 **Optional:** to measure *calculating* time (work your app does before a request, such as building a large payload), wrap that work. This helper still runs your code if the script is blocked by an ad blocker:

requestTiming.ts

```
declare global {
  interface Window {
    RequestTiming?: { measure<T>(fn: () => T): T; markCalculating(): void; flush(): void };
  }
}

export const measure = <T,>(fn: () => T): T =>
  window.RequestTiming ? window.RequestTiming.measure(fn) : fn();

// usage
await measure(() => {
  const body = buildPayload();        // counted as "calculating"
  return api.post('/orders', body);   // the first request it sends
});
```

Mobile developer In a mobile app

Flutter, React Native, Swift, Kotlin: there is no script to load, so the app reports the timestamps itself. It's one small wrapper around the HTTP client.

Step	What the app does
Before the call	Note the time in epoch milliseconds: `t0`.
After the response	Note the time again: `t3`. Read the `X-Request-Timing` response header: `id=…;t1=…;t2=…;n=…`. Not every response has it (the server samples), so skip the ones without.
Clock offset	Per server `n`, keep the offset from the fastest exchange: `((t1 - t0) + (t2 - t3)) / 2`, where fastest means the smallest `(t3 - t0) - (t2 - t1)`. Refresh it every 10 minutes.
Report	Every few seconds (or at 20 entries), POST up to 50 entries as JSON to `/performance-tracking/api/client`. Also send when the app goes to the background.

Flutter (package:http)

request_timing.dart

```
 import 'dart:async';
import 'dart:convert';
import 'package:http/http.dart' as http;

class RequestTiming {
  RequestTiming._();
  static final instance = RequestTiming._();

  static final _endpoint = Uri.parse('https://api.example.com/performance-tracking/api/client');
  static const _offsetTtlMs = 10 * 60 * 1000;

  final _queue = <Map<String, Object?>>[];
  final _offsets = <String, ({double offset, double rtt, double at})>{};
  Timer? _timer;

  static double _now() ⇒ DateTime.now().microsecondsSinceEpoch / 1000;

  /// Use instead of client.send() for calls to the API. Reads the whole body.
  Future<http.Response> send(http.Client client, http.BaseRequest request, {double? calcMs}) async {
    final t0 = _now();
    final response = await http.Response.fromStream(await client.send(request));
    final t3 = _now();
    final header = response.headers['x-request-timing'];
    if (header ≠ null) _record(header, t0, t3, calcMs);
```

```
    return response;
  }

  void _record(String header, double t0, double t3, double? calcMs) {
    final parts = <String, String>{
      for (final p in header.split(';'))
        if (p.contains('=')) p.substring(0, p.indexOf('=')).trim(): p.substring(p.indexOf('=') + 1).trim(),
    };
    final id = parts['id'];
    final t1 = double.tryParse(parts['t1'] ?? '');
    final t2 = double.tryParse(parts['t2'] ?? '');
    if (id == null || t1 == null || t2 == null) return;

    // Clock offset (server minus device), per server, from the fastest exchange.
    final node = parts['n'] ?? '';
    final rtt = (t3 - t0) - (t2 - t1);
    final best = _offsets[node];
    if (rtt >= 0 && (best == null || rtt <= best.rtt || t3 - best.at > _offsetTtlMs)) {
      _offsets[node] = (offset: ((t1 - t0) + (t2 - t3)) / 2, rtt: rtt, at: t3);
    }

    _queue.add({
      'id': id, 't0': t0, 't3': t3,
      'offset_ms': _offsets[node]?.offset,
      if (calcMs != null) 'calc_ms': calcMs,
    });
    if (_queue.length >= 20) {
      flush();
    } else {
      _timer ??= Timer(const Duration(seconds: 5), flush);
    }
  }

  /// Also call this when the app goes to the background.
  Future<void> flush() async {
    _timer?.cancel();
    _timer = null;
    if (_queue.isEmpty) return;
    final batch = _queue.take(50).toList();
    _queue.removeRange(0, batch.length);
    try {
      await http.post(_endpoint,
          headers: {'Content-Type': 'application/json'},
          body: jsonEncode({'timings': batch}));
    } catch (_) {} // monitoring never breaks the app
    if (_queue.isNotEmpty) await flush();
  }
}

// usage
final response = await RequestTiming.instance.send(
  client, http.Request('GET', Uri.parse('https://api.example.com/api/orders')));
```

React Native (or any JavaScript without a browser page)

requestTiming.ts

```
 const ENDPOINT = 'https://api.example.com/performance-tracking/api/client';
```

```ts
const OFFSET_TTL_MS = 10 * 60 * 1000;

type Report = { id: string; t0: number; t3: number; offset_ms: number | null; calc_ms?: number; headers_only:
boolean };

const queue: Report[] = [];
const offsets = new Map<string, { offset: number; rtt: number; at: number }>();
let timer: ReturnType<typeof setTimeout> | null = null;

/** Use instead of fetch() for calls to the API. */
export async function timedFetch(input: RequestInfo, init?: RequestInit, calcMs?: number): Promise<Response> {
  const t0 = Date.now();
  const response = await fetch(input, init);
  const header = response.headers.get('x-request-timing');
  if (header) record(header, t0, Date.now(), calcMs);
  return response;
}

function record(header: string, t0: number, t3: number, calcMs?: number) {
  const parts: Record<string, string> = {};
  for (const part of header.split(';')) {
    const i = part.indexOf('=');
    if (i > 0) parts[part.slice(0, i).trim()] = part.slice(i + 1).trim();
  }
  const t1 = Number(parts.t1);
  const t2 = Number(parts.t2);
  if (!parts.id || !Number.isFinite(t1) || !Number.isFinite(t2)) return;

  // Clock offset (server minus device), per server, from the fastest exchange.
  const node = parts.n ?? '';
  const rtt = (t3 - t0) - (t2 - t1);
  const best = offsets.get(node);
  if (rtt ⩾ 0 && (!best || rtt ⩽ best.rtt || t3 - best.at > OFFSET_TTL_MS)) {
    offsets.set(node, { offset: ((t1 - t0) + (t2 - t3)) / 2, rtt, at: t3 });
  }

  queue.push({
    id: parts.id, t0, t3,
    offset_ms: offsets.get(node)?.offset ?? null,
    ...(calcMs ≠≠ undefined ? { calc_ms: calcMs } : {}),
    headers_only: true, // t3 is when the headers arrived; the caller reads the body later
  });
  if (queue.length ⩾ 20) void flush();
  else timer ??= setTimeout(() ⇒ void flush(), 5000);
}

/** Also call this from an AppState 'background' listener. */
export async function flush(): Promise<void> {
  if (timer) clearTimeout(timer);
  timer = null;
  const batch = queue.splice(0, 50);
  if (!batch.length) return;
  try {
    await fetch(ENDPOINT, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ timings: batch }),
    });
  } catch { /* monitoring never breaks the app */ }
```

```
  if (queue.length) await flush();
}
```

Swift and Kotlin follow the same four steps: read the header after the call, keep the offset per server, and post the batch as JSON. The endpoint answers with `{"accepted": n, "rejected": n}`; reports more than 10 minutes old, or sent twice, are rejected.

Everyone Check that it works

1. **The header is there.** In the browser's devtools, open the Network tab and click an API call. Its response headers should include `X-Request-Timing`. The server records only a sample of requests, so check a few. From a terminal:

   ```terminal
   curl -s -D - -o /dev/null -H "Origin: https://app.example.com" https://api.example.com/api/your-endpoint \
     | grep -i -E 'x-request-timing|access-control-expose-headers|timing-allow-origin'
   ```

   For the web frontend, the response also needs `Access-Control-Expose-Headers: X-Request-Timing`. If it is missing, the API owner's step 1 hasn't taken effect yet.

2. **Reports are sent.** About two seconds after a call, the Network tab shows a request to `/performance-tracking/api/client` (type *ping* or *fetch*) that answers with `accepted` above 0. In a mobile app, log the response of `flush()`.

3. **The dashboard fills in.** At `/performance-tracking` on the API, rows without a `*` have client timings. Open one to see network out and network return in its waterfall.

Don't cache API responses publicly (`Cache-Control: public` or a CDN cache). A cached response carries another request's timing id, and its reports are rejected.